EXPRESS.JS

Assignment Answers – Questions 1 to 10

Concepts • Routing • Middleware • JSON • CRUD API • Postman Testing

Technology: Node.js and Express.js
Server Port: 3000

1. What is Express.js? Why is it commonly used with Node.js?

Express.js is a lightweight and flexible web application framework for Node.js. It is used to build web servers, web applications and REST APIs.

Reasons for using Express.js:
• Simplifies server creation and request handling.
• Provides routing for URLs and HTTP methods such as GET, POST, PUT and DELETE.
• Supports middleware for logging, authentication and validation.
• Makes REST API development easier.
• Can connect to databases such as MongoDB and MySQL.

Node.js provides the JavaScript runtime, while Express.js provides convenient web-server and routing features.

2. Create a basic Express.js application on port 3000

In VS Code terminal, create the project and install Express:

```
npm init -y
npm install express
```

Create **server.js**:

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
    res.send('Welcome to Express.js');
});

app.listen(3000, () => {
    console.log('Server running on port 3000');
});
```

Run **node server.js** and open **http://localhost:3000/**. Expected output: Welcome to Express.js.

3. Create routes for /, /about, /contact and /students

Create a JavaScript file named **server.js** (or **question3.js** if each question is in a separate file).

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
    res.send('Welcome to Home Page');
});

app.get('/about', (req, res) => {
    res.send('This is About Page');
});

app.get('/contact', (req, res) => {
    res.send('Contact Us');
});

app.get('/students', (req, res) => {
    res.json({ message: 'Student Details' });
});

app.listen(3000, () => {
    console.log('Server started on port 3000');
});
```

Routes and expected responses:
GET / → Welcome to Home Page
GET /about → This is About Page
GET /contact → Contact Us
GET /students → { message: 'Student Details' }

4. Create GET /students/:id route

A route parameter is a dynamic value in the URL. Express makes it available through **req.params**.

```
const express = require('express');
const app = express();

app.get('/students/:id', (req, res) => {
    const studentId = req.params.id;
    res.json({
        message: 'Student ID Received',
        id: studentId
    });
});

app.listen(3000, () => {
    console.log('Server running on port 3000');
});
```

Request: GET http://localhost:3000/students/101

Response:

```
{
    "message": "Student ID Received",
    "id": "101"
}
```

5. Create GET /students using a course query parameter

A query parameter is supplied after ? in the URL. Express reads it through **req.query**.

```
const express = require('express');
const app = express();

app.get('/students', (req, res) => {
    const course = req.query.course;
    res.json({
        message: 'Students Course Details',
        course: course || 'All Courses'
    });
});

app.listen(3000, () => {
    console.log('Server running on port 3000');
});
```

Request: GET http://localhost:3000/students?course=MERN

Response:

```
{
    "message": "Students Course Details",
    "course": "MERN"
}
```

6. Create POST /students endpoint

POST sends data from the client to the server. The endpoint reads name, age, course and marks from **req.body**.

```
const express = require('express');
const app = express();

app.use(express.json());

app.post('/students', (req, res) => {
    const { name, age, course, marks } = req.body;
    res.status(201).json({
        message: 'Student Added Successfully',
        student: { name, age, course, marks }
    });
});

app.listen(3000, () => {
    console.log('Server running on port 3000');
});
```

In Postman, send POST http://localhost:3000/students with Body → raw → JSON:

```
{
    "name": "Arun",
    "age": 20,
    "course": "MERN",
    "marks": 85
```

```
}
```

The response returns the received student data with status **201 Created**.

7. Explain middleware and create a request-logging middleware

Middleware is a function that runs during the request-response cycle. It can inspect or modify the request and response, end the request, or pass control using **next()**. Uses include logging, authentication, validation and error handling.

```
const express = require('express');
const app = express();

const logger = (req, res, next) => {
    console.log('Method:', req.method);
    console.log('URL:', req.url);
    next();
};

app.use(logger);

app.get('/', (req, res) => {
    res.send('Home Page');
});

app.get('/about', (req, res) => {
    res.send('About Page');
});

app.listen(3000, () => {
    console.log('Server running on port 3000');
});
```

For a request to /about, the terminal logs Method: GET and URL: /about.

8. Use express.json() and explain why it is required

express.json() is built-in middleware that parses incoming JSON request bodies. The parsed object becomes available through **req.body**. Register it before the route.

```
const express = require('express');
const app = express();

app.use(express.json());

app.post('/student', (req, res) => {
    console.log(req.body);
    res.json({
        message: 'Data Received',
        data: req.body
    });
});

app.listen(3000, () => {
    console.log('Server running on port 3000');
});
```

Send POST http://localhost:3000/student with Body → raw → JSON:

```
{
    "name": "Kumar",
    "age": 21
}
```

Without JSON-parsing middleware (or another suitable body parser), req.body is normally undefined for JSON requests.

9. Create a products CRUD API using a temporary array

CRUD means Create, Read, Update and Delete. Products are stored in memory in an array, so data resets when the server restarts.

```
const express = require('express');
const app = express();

app.use(express.json());

let products = [
    { id: 1, name: 'Laptop', price: 50000 },
```

```
    { id: 2, name: 'Mouse', price: 500 }
];

// GET - Read all products
app.get('/products', (req, res) => {
    res.json(products);
});

// GET - Read one product
app.get('/products/:id', (req, res) => {
    const product = products.find(
        p => p.id === Number(req.params.id)
    );
    if (!product) {
        return res.status(404).json({ message: 'Product not found' });
    }
    res.json(product);
});

// POST - Create a product
app.post('/products', (req, res) => {
    const { name, price } = req.body;
    if (!name || price === undefined) {
        return res.status(400).json({
            message: 'name and price are required'
        });
    }

    const newProduct = {
        id: products.length
            ? Math.max(...products.map(p => p.id)) + 1
            : 1,
        name,
        price
    };
    products.push(newProduct);
    res.status(201).json(newProduct);
});

// PUT - Update a product
app.put('/products/:id', (req, res) => {
    const product = products.find(
        p => p.id === Number(req.params.id)
    );
    if (!product) {
        return res.status(404).json({ message: 'Product not found' });
    }

    const { name, price } = req.body;
    if (name !== undefined) product.name = name;
    if (price !== undefined) product.price = price;

    res.json({
        message: 'Product Updated Successfully',
        product
    });
});

// DELETE - Remove a product
app.delete('/products/:id', (req, res) => {
    const index = products.findIndex(
        p => p.id === Number(req.params.id)
    );
    if (index === -1) {
        return res.status(404).json({ message: 'Product not found' });
    }

    products.splice(index, 1);
    res.json({ message: 'Product Deleted Successfully' });
});

app.listen(3000, () => {
    console.log('Server running on port 3000');
});
```

CRUD endpoints:

GET /products – retrieve all products

GET /products/1 – retrieve product with ID 1

POST /products – add a product

PUT /products/1 – update product with ID 1

DELETE /products/1 – delete product with ID 1

10. Test the Express.js API using Postman

Postman is an API testing tool used to send HTTP requests and inspect responses.

Testing steps:
1. Start the server using node server.js.
2. Open Postman and create a new HTTP request.
3. Select the HTTP method and enter the URL.
4. For POST and PUT, choose Body → raw → JSON and enter the request data.
5. Click Send and inspect the status code and response body.
6. Capture a screenshot of each request and response for submission.

Suggested test sequence:

No.	Method and URL	Body	Expected status
1	GET /products	None	200 OK
2	GET /products/1	None	200 OK
3	POST /products	{"name":"Keyboard","price":1500}	201 Created
4	PUT /products/3	{"name":"Gaming Keyboard","price":2500}	200 OK
5	DELETE /products/3	None	200 OK

Run these requests in order. The sample POST creates ID 3 when the original array contains IDs 1 and 2; therefore, the PUT and DELETE examples use /products/3. Adjust the ID if your data differs.

Screenshot note: These are expected sample results, not actual Postman screenshots. Execute the requests in Postman and insert your own screenshots into the assignment.

Project Setup and GitHub Submission

Suggested folder structure for separate question files:

```
Express-JS-Assignment/
|-- Question-2/server.js
|-- Question-3/server.js
|-- Question-4/server.js
|-- Question-5/server.js
|-- Question-6/server.js
|-- Question-7/server.js
|-- Question-8/server.js
|-- Question-9/server.js
|-- package.json
|-- package-lock.json
|-- .gitignore
`-- README.md
```

Use the **.js** extension for JavaScript files. If multiple files listen on port 3000, stop the previous server with Ctrl+C before starting another. Alternatively, combine the routes in one server.js file.

Install Express:

```
npm init -y
npm install express
```

.gitignore should contain:

```
node_modules/
```

GitHub commands (replace the placeholder with your own repository URL):

```
git init
git add .
git commit -m "Added Express.js assignment"
git branch -M main
git remote add origin YOUR_REPOSITORY_URL
git push -u origin main
```

Result

Thus, the Express.js examples demonstrate server creation, routing, route parameters, query parameters, POST request handling, middleware, JSON parsing and CRUD operations. The API endpoints can be tested using Postman.